Opposition Agents Playing Magic: The Gathering

Knowledge-Graph-Grounded JEPA World Models and Active-Inference LLM Agents

Felix Neubürger[1] and Jonas Neubürger[1]

[1]Independent Research , github.com/cuteredpwnda/opposition-agents-playing-mtg

April 2026

Abstract

Magic: The Gathering (MTG) is, by several measures, the most combinatorially complex commercial card game ever published: more than 28,000 unique cards, a 260-page rule book, partial observability over 50–90 hidden cards, an unbounded action space, a turn-based stack with priority, and a state space conservatively estimated above 10^{800}. We present Opposition Agents Playing MTG, an open-source research framework that combines (i) a from-scratch implementation of the Comprehensive Rules game engine, (ii) a Neo4j property-graph database holding a knowledge graph (KG) backed by an OWL 2 (Web Ontology Language, version 2) ontology of cards, mechanics, archetypes and combos, (iii) a four-component world model in the Vision–Memory–Controller (V+M+C) tradition of Ha & Schmidhuber (2018) extended with a Joint-Embedding Predictive Architecture (JEPA) head in the spirit of LeCun (2022), and (iv) Active-Inference Large-Language-Model (LLM) agents that fuse symbolic graph queries, latent imagination and free-energy-minimising action selection. The framework treats one seat of an MTG game as a Partially Observable Markov Decision Process (POMDP) embedded inside an n-player Markov game, and we derive each agent as an explicit approximation to that POMDP. We describe the system end-to-end, justify the design choices against the structure of the game, and report on a champion-vs-challenger self-play promotion loop with Elo (Elo, 1978) rating tracking. The code, ontology, and this report are released under the Massachusetts Institute of Technology (MIT) licence.

Keywords: world models, JEPA, knowledge graphs, active inference, LLM agents, self-play, neuro-symbolic reasoning, Magic: The Gathering.

1 Introduction

Reinforcement-learning (RL) research on games has historically progressed through a sequence of increasingly hard environments: backgammon (Tesauro, 1995), chess and Go (Silver et al., 2017, 2018), StarCraft II (Vinyals et al., 2019), and no-limit Texas hold'em poker (Brown & Sandholm, 2018). Each step pushed the community to invent new tools — temporal-difference (TD) learning, deep value networks, self-play, counterfactual regret minimisation (CFR). *Magic: The Gathering* (MTG) sits beyond the union of these challenges (Table 1): its action space is unbounded, its rules are open-textured natural language printed on cards (the comprehensive rule book itself defers to the per-card text in many places), its hidden information includes the entire opponent's deck composition, and its mechanics are extended every quarter by the publisher Wizards of the Coast. A single MTG card can override the rules of the game (e.g. *Yixlid Jailer*, *Blood Moon*, *Cumulative Upkeep*), and a single decision in combat may resolve dozens of triggered abilities in a player-chosen order. The state space is open in the strict sense:

Table 1: Approximate combinatorial complexity comparison.

	Chess	**Go**	**No-limit Poker (HU)**	**MTG**
State space	10^{47}	10^{170}	10^{160}	$\mathbf{> 10^{800}}$
Branching factor	∼35	∼250	∼10	0–100^{+} (unbounded)
Hidden information	none	none	2 cards	50–90+ cards
Card / piece pool	6 types	1 type	52 cards	>28,000 unique
Rule text length	∼10p	∼5p	∼10p	∼260p + per-card
Rules update frequency	decades	decades	decades	quarterly

every quarterly set adds previously-unseen card text whose semantics must be parsed at runtime, so any agent that hard-codes its action vocabulary cannot survive a set rotation.

For a concrete sense of why this matters for an algorithmic agent: in a single midgame turn the active player may need to (i) decide whether to play a land, (ii) sequence two or three spells with mana cost, colour requirements and stack ordering, (iii) optionally activate any number of permanent abilities that themselves go on the stack, (iv) declare attackers under summoning-sickness and tapping rules, (v) respond to triggered abilities resolving in the order chosen by their controllers, and (vi) decide which damage to assign in combat under multi-blocker rules. Each of these is itself a sub-decision tree. The total number of legal actions per priority pass routinely exceeds 60 and, in pathological combo states, several hundred. A model-free policy that emits a fixed-size logit vector cannot represent this branching, and a tabular search procedure that enumerates the subtree cannot fit it into memory.

A purely model-free deep-RL approach is therefore unattractive: the action space cannot be enumerated as a fixed-size logit vector, the observation space is symbolic and irregular, and every new set introduces previously-unseen card text whose semantics must be parsed at run time. Conversely, a purely symbolic LLM-prompting approach struggles with rules-correctness, hallucinated cards, and the long-horizon credit-assignment that competitive play requires — an LLM that simply names a move it would like to make has no guarantee that move is legal, that it resolves the way the LLM expects, or that it furthers a coherent multi-turn plan.

Our position is that the structure of MTG demands a *neuro-symbolic* stack: a verified rules engine to keep play legal, a knowledge graph to make card semantics queryable, a learned world model to enable imagination and planning over the resulting compressed state, and an LLM agent that orchestrates these components under an active-inference objective. Figure 1 sketches the resulting architecture as a data-flow diagram, separating the *symbolic* substrate (data sources, engine, knowledge graph) from the *learned* substrate (world model, agent zoo) and showing where information actually crosses the boundary.

Contributions. This work makes four contributions.

1. A *from-scratch* Comprehensive-Rules MTG engine in Python with stack/priority, layered continuous effects, replacement effects, triggers and state-based actions, exposing a deterministic step API suitable for batched self-play.
2. An *ontology-grounded knowledge graph* of cards, mechanics, archetypes and combos, imported into Neo4j via the `n10s` OWL importer and made queryable from agents through a thin GraphRAG layer.
3. A *V+M+C+JEPA* world model where the state encoder fuses tokenised game state with a graph-context vector from the KG, the dynamics model is an MDN-LSTM over the latent, and a JEPA head provides a contrastive predictor for surprise scoring and dream rollouts.

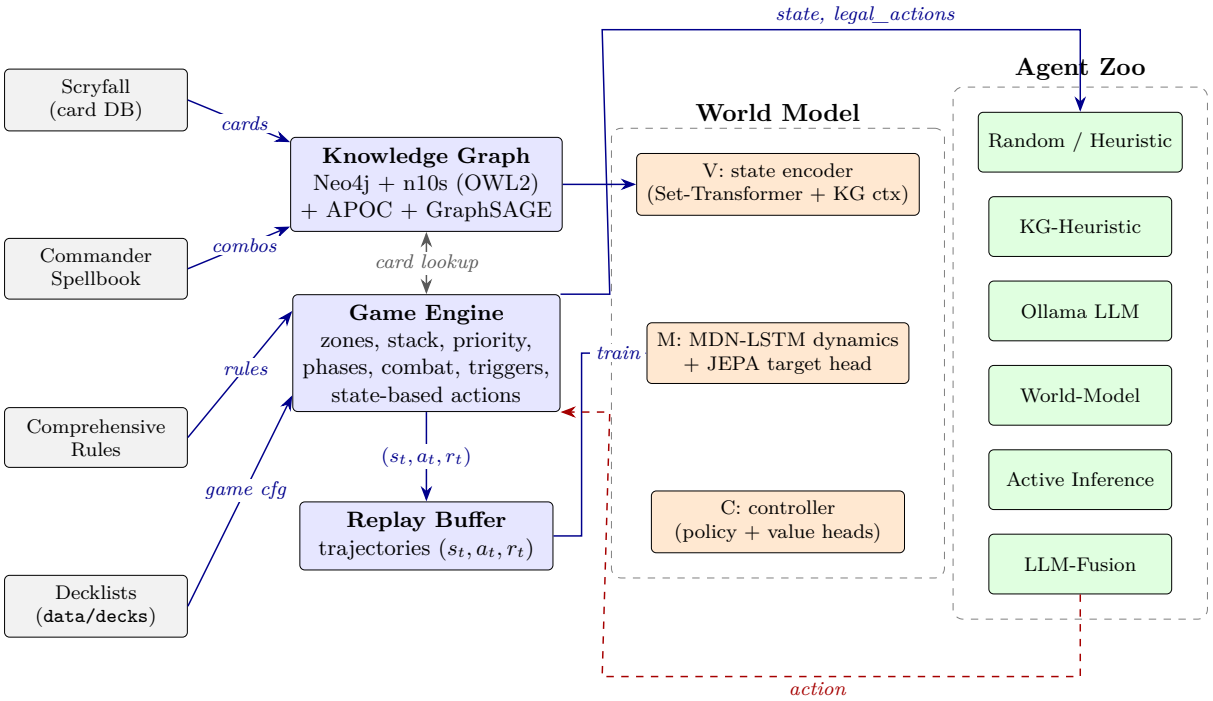

Figure 1: Data-flow architecture. **Grey** boxes are external data sources; **blue** the symbolic substrate (engine, knowledge graph, replay buffer); **orange** the learned world model (V+M+C with a JEPA target head and a knowledge-graph context channel); **green** the agent zoo. Solid blue arrows are forward data flow; the red dashed arrow is the agent's chosen action returned to the engine; the engine ↔ knowledge-graph link is bidirectional (card lookup at decision time, ontology-grounded card metadata at engine boot). Each agent additionally has read-only access to the knowledge graph and to the world-model controller through a thin lookup interface (omitted from the figure for clarity, described in the body text). The engine is the only component allowed to mutate game state; agents read but do not write.

4. An *active-inference LLM-fusion agent* that chooses actions by minimising expected free energy (EFE) combining pragmatic value, epistemic value, and a calibrated LLM prior; a self-play training loop with Elo rating, opponent-pool sampling and champion-vs-challenger promotion is provided.

Why these four together. Each contribution targets a specific failure mode of the others. The engine prevents the LLM from playing illegal moves and the world model from imagining inconsistent dynamics. The knowledge graph compresses the 28,000-card pool into a structure with a few hundred archetype clusters so that an agent can reason about "what card type does this state want next" without enumerating all candidates. The world model amortises the cost of multi-turn lookahead so that decisions remain well within a few hundred milliseconds even when planning ten turns deep. The active-inference fusion agent finally ties the three together with a single tunable objective — expected free energy — whose pragmatic and epistemic terms have a direct mapping onto MTG concepts of *value* and *information*.

Scope and what we do *not* do. We do not solve MTG; we do not even target tournament-strength play. The contribution is a research substrate that is rules-correct on the implemented card pool, that maintains a queryable symbolic representation of card semantics, and that supports plug-in research on world models and inference-style agents on a problem qualitatively harder than the standard benchmarks. The empirical claims in §6 are therefore framework-level (decks complete, agents converge faster than baselines, ablations show graded contributions)

rather than absolute strength claims against human play.

The framework is released as open source[1] and the codebase is the canonical artifact accompanying this report. Section 2 positions the work against prior literature; §3 formalises the POMDP and the four learning principles (world models, FEP, knowledge-graph priors, constrained-decoding LLMs); §4 walks the architecture top-down; §6 presents the empirical protocol and preliminary results; §7–8 discuss limitations and next steps.

2 Related Work

World models. Ha & Schmidhuber (2018) introduced the V+M+C decomposition — a vision-VAE, a recurrent dynamics model with mixture-density output, and a tiny linear/MLP controller trained in the dream of M. Successors include Dreamer (Hafner et al., 2020, 2023) and the Schmidhuber-line of self-referential systems (Schmidhuber, 2015; Irie et al., 2022). We adopt the V+M+C skeleton but (i) replace the convolutional vision encoder with a Set-Transformer over symbolic game tokens, and (ii) attach a JEPA head (LeCun, 2022; Assran et al., 2023) that predicts a target encoding rather than a pixel-level reconstruction — a closer fit to MTG, where "imagination" should preserve abstract game state, not card art.

Active inference. The Free-Energy Principle of Friston (2010) and its discrete-state operationalisation (Sajid et al., 2021) provide a principled objective for action selection under uncertainty: minimise expected free energy, which decomposes into a pragmatic (reward-seeking) and an epistemic (information-seeking) term. We use this decomposition to balance "play to win" against "probe the opponent" decisions during the early game.

Knowledge graphs and GraphRAG. Neo4j with the `n10s` plugin enables direct ingestion of OWL ontologies as labeled property graphs (Neo4j Labs, n.d.); the APOC library adds graph algorithms; GraphSAGE (Hamilton et al., 2017) provides inductive node embeddings. GraphRAG (Edge et al., 2024) has recently emerged as a complement to vector RAG for queries that benefit from explicit relational structure. MTG is a textbook GraphRAG case: "find me a two-card combo using only cards in my library that survive Counterspell" is a multi-hop graph query, not a similarity search.

MTG and other complex card games. Earlier MTG-AI work has used hand-crafted heuristics (Cowling et al., 2012), MCTS, and small policy nets on restricted card pools (e.g., `open-mtg`). On the rules side, `Forge`, `XMage` and `Mage` provide JVM-based reference implementations. To our knowledge, no prior public system combines a full Comprehensive-Rules engine with a learned world model and a graph-grounded LLM controller. Moravčík et al. (2017) (poker), Vinyals et al. (2019) (StarCraft) and Schrittwieser et al. (2020) (MuZero) are spiritual antecedents at the algorithmic level.

Ontologies for games. Game ontologies have been explored for serious games and educational metadata; for trading-card games specifically, public OWL ontologies are sparse. Our ontology (`data/ontology/mtg-ontology-v1.1.owl`) is hand-crafted following the methodology surveyed in Noy & McGuinness (2001) and validated by SHACL shapes (Knublauch & Kontokostas, 2017). The framework is designed to interoperate with the OntologyExtender tool (Data Science Lab FHSWF, 2024) — a separate, in-development multi-agent human-in-the-loop ontology-evolution toolchain by the same lab — but in the present version the OWL schema is a fixed v1.1 artefact: we do not currently run automated extension passes.

[1] `https://github.com/cuteredpwnda/opposition-agents-playing-mtg`

3 Theoretical Foundations

3.1 MTG as a partially-observable Markov game

We model a game of Magic from the perspective of a single seat as a finite-horizon POMDP $(\mathcal{S}, \mathcal{A}, \mathcal{O}, T, \Omega, R, \gamma, b_0)$ embedded inside an n-player Markov game. The full state $s_t \in \mathcal{S}$ encompasses every public and private zone of every player — libraries, hands, the stack, mana pools, and a long tail of bookkeeping flags (summoning sickness, monarchy, the day/night state, designations, emblems, dungeon room, companion eligibility). A controlled-perspective filter $\Omega : \mathcal{S} \to \mathcal{O}$ hides the opposing players' hands and the order of their libraries; the agent only ever observes $o_t = \Omega(s_t)$. The action space $\mathcal{A}_t = \mathrm{LEGAL}(s_t)$ is computed afresh on every priority pass and is highly variable: typically $|\mathcal{A}_t| \in [1, 60]$, but can spike past several hundred in hard combo states. The transition model $T(s' \mid s, a)$ is the rules engine itself, fully deterministic given a random seed and the active player's choices. Reward R is sparse and terminal — ± 1 on the win condition — but agents may shape it during training using Δlife, Δcards, Δtempo as auxiliary signals. The state space is combinatorially large: a conservative count over deck-times-zone configurations gives $|\mathcal{S}| > 10^{800}$, several orders of magnitude beyond Go.

The optimal value function over beliefs $b_t \in \Delta(\mathcal{S})$ satisfies the Bellman equation

$$V^*(b) = \max_{a \in \mathcal{A}} \left[\sum_s b(s)\, R(s,a) + \gamma \sum_o \Pr(o \mid b, a)\, V^*(b'_o) \right], \tag{1}$$

where b'_o is the Bayes update of b after taking a and observing o. Exact solution is intractable. Each agent in our zoo is therefore an explicit approximation that trades different axes of accuracy: tabular reactive policies (Random, Heuristic, KG-Heuristic, LLM) collapse b_t to a point estimate $s_t \approx o_t$; the world-model agent learns an amortised Bellman residual in latent space; the active-inference agents maintain an explicit belief and minimise expected free energy.

3.2 World models as latent imagination

The V+M+C decomposition of Ha & Schmidhuber (2018) factors an agent into a perception module V, a recurrent dynamics module M, and a controller C. We follow this skeleton but adopt LeCun and Assran's recommendation (LeCun, 2022; Assran et al., 2023) to predict in *representation space* rather than in observation space: the model is trained to match its H-step rollout of the latent against a target encoder's output, not to reconstruct cards or board art. Concretely the training loss for the dynamics is

$$\mathcal{L}_{\mathrm{JEPA}} = \sum_t \left\| \hat{z}_{t+1} - \mathrm{sg}(z_{t+1}) \right\|_2^2 + \beta \cdot \mathrm{KL}\big(q(z_t \mid o_t) \,\|\, p(z_t \mid h_{t-1})\big), \tag{2}$$

where sg is stop-gradient on the target branch, h_{t-1} is the previous LSTM hidden state of M, and β is the single tunable hyperparameter we expose (`-jepa-beta`, default 1.0). At decision time the controller scores each candidate action by a $K \times H$ Monte-Carlo rollout in latent space:

$$\mathrm{score}(a) = \frac{1}{K} \sum_{k=1}^{K} V_\theta\Big(h^{(k)}_{t+H} \,\Big|\, a_t = a\Big), \tag{3}$$

with $K = 8$, $H = 10$ by default. No actual rules-engine simulation is performed during inference; this is what makes the agent fast despite the engine being heavy. The controller therefore amortises the Bellman residual: C is implicitly approximating $\arg\max_a Q^*(s_t, a)$ given an imperfect $\hat{T}$.

3.3 Active inference and the free-energy principle

Friston's free-energy principle (Friston, 2010) frames perception and action as two sides of variational inference under a generative model $p(o, s)$. Variational free energy decomposes as

$$F[q] = \mathbb{E}_q[\log q(s) - \log p(o, s)] = \mathrm{KL}(q(s) \,\|\, p(s \mid o)) - \log p(o), \tag{4}$$

so minimising F over q simultaneously sharpens beliefs (the KL term) and accumulates evidence ($-\log p(o)$). Action selection is recovered by minimising *expected* free energy, conditioned on a policy $\pi = (a_t, a_{t+1}, \dots)$:

$$G(\pi) = \underbrace{-\,\mathbb{E}_q[\log p(o_\tau \mid C)]}_{\text{pragmatic value}} + \underbrace{-\,\mathbb{E}_q\big[\mathrm{KL}(q(s_\tau \mid o_\tau, \pi) \,\|\, q(s_\tau \mid \pi))\big]}_{\text{epistemic value}}, \tag{5}$$

where C encodes the agent's preferences (in our setting: "win"), $q(s_\tau \mid \pi)$ is the predictive belief under policy π, and the epistemic term rewards information-gathering actions. Bayesian RL falls out as the special case where preferences are uniform; pure exploration falls out when rewards vanish. In the MTG context, the pragmatic term encodes "play to win" and the epistemic term encodes "probe the opponent's hand" — exactly the trade-off that Sajid et al. (2021) formalise. Our `ActiveInferenceAgent` maintains a Dirichlet belief over the opponent's hand composition seeded from the decklist (cf. `docs/OPPONENT_MODELING_WITH_DECKLIST.md`) and re-evaluates Eq. (5) at every priority pass.

3.4 Knowledge graphs as a symbolic prior

Cards form an inductive graph $\mathcal{G} = (V, E, \tau)$ with typed edges $\tau \in \{\text{combo}, \text{synergy}, \text{counter}, \text{enables}\}$ ingested from the OWL ontology and an embedding pipeline trained with GraphSAGE (Hamilton et al., 2017). GraphSAGE is inductive: a freshly printed card v_{new} obtains an embedding $\mathbf{h}_{v_{\text{new}}} \in \mathbb{R}^{128}$ by sampling and aggregating its neighbours, without retraining. We exploit this in two ways: (i) the encoder feeds a graph latent $g_t = \mathrm{Enc}_{KG}(\mathcal{G}, s_t)$ into the recurrent dynamics so that the world model's prior is graph-aware, and (ii) the heuristic agent rescores candidate spells by their graph distance to combo completions, mirroring GraphRAG's "relational evidence" pattern (Edge et al., 2024).

3.5 LLM as a constrained-decoding policy

The LLM never freely generates moves. Given the prompt $\rho(s_t, \mathcal{A})$ that enumerates legal actions by index, we read off

$$\log p_{\text{LLM}}(a \mid s_t) = \mathrm{LM}(\mathrm{idx}(a) \mid \rho(s_t, \mathcal{A})), \tag{6}$$

masking any token outside the integer range $\{0, \dots, |\mathcal{A}| - 1\}$. This recovers the standard "constrained decoding" guarantee — the LLM cannot hallucinate an illegal move — while still letting it import its general-knowledge prior. Calibration of the resulting distribution is controlled by a temperature parameter (0.2 in our default config), which interpolates between greedy LLM picks and a uniform fallback.

3.6 Self-play promotion as fixed-point iteration

Training combines all three signals. The replay buffer $\mathcal{D}$ is collected by self-play between agent variants; the world model is fit on $\mathcal{D}$; and the controller is improved iteratively by the champion-vs-challenger schema implemented in `scripts/train_pipeline.py`. At each iteration i, a challenger π_{i+1} is trained against π_i on the latest replay window and faces the champion in a best-of-N match; promotion happens iff the empirical win rate exceeds 0.55. This is a stochastic-approximation fixed-point operator on the policy improvement step (Tesauro, 1995;

Silver et al., 2017, 2018; Schrittwieser et al., 2020): under standard Robbins–Monro conditions it converges, in expectation, to a fixed point of the Bellman optimality operator restricted to the dynamics class $\hat{T}$ that the world model has learned. Convergence in practice depends on $\hat{T}$ being locally accurate, which we monitor by tracking JEPA validation loss across iterations.

4 System Architecture

4.1 Game engine

The engine implements the Comprehensive Rules subset relevant for two-player Standard plus the structural extensions for multiplayer Commander. Two parallel orchestrators sit on top of the same dataclasses: a synchronous `GameSimulator` (`src/engine/game_simulator.py`) used in tournaments and tests, and an asynchronous `GameRunner` (`src/orchestrator/game_runner.py`) used in self-play data collection. Both implement the same end-of-game rules. Core modules:

- `game_state`: immutable-by-convention dataclasses for `GameState`, `PlayerState` (carrying `max_hand_size` and `mulligans_taken`), and `CardInstance`;
- `zones`, `stack`, `phases`, `priority_loop`: APNAP priority and the seven-step turn structure;
- `rules_engine`: legal-action generation given current priority and phase;
- `state_based_actions` (CR 704), `triggered_abilities` (CR 603), `replacement_effects` (CR 614), `continuous_effects` (CR 613 layer system);
- `combat`, `mana`, `keywords`, `abilities`: per-mechanic logic;
- **London mulligan** at game setup: draw 7, optionally re-mulligan up to a configurable cap, then bottom cards equal to the number of mulligans taken (CR 103.5);
- **Cleanup-phase discard** down to `PlayerState.max_hand_size` (default 7), which had been a long-standing gap in earlier engine snapshots (CR 514);
- **Deterministic empty-library loss**: drawing from an empty library ends the game with that player losing (CR 104.3c / 704.5b), in both the sync simulator and the async runner;
- **Deterministic timeout tie-breaker**: instead of auto-draw on hitting the configured turn limit, the leader on the lexicographic tuple (life total, permanents in play, hand size, library size) wins; only true ties are recorded as draws.

The engine is pure-Python, deterministic given an RNG seed, and has no LLM dependency, which makes it the ground truth that all agents play against.

Table 2 maps each Comprehensive-Rules concept to the exact module that implements it; readers reproducing the system can use it as a reading order through the codebase.

4.2 Knowledge graph

Cards are imported from Scryfall and reified as `(:Card)` nodes carrying mana cost, type line, oracle text, colour identity, and a SBERT embedding of the oracle text. Mechanics, abilities, archetypes and combos become first-class nodes connected by typed edges (`HAS_KEYWORD`, `ENABLES_COMBO`, `ARCHETYPE_OF`, ...). The OWL ontology (v1.1) declares the schema; `n10s` imports it; `pyshacl` validates it; APOC provides PageRank, community detection and shortest-path queries used in agent reasoning. **??** 1 shows a typical multi-hop query used by the combo-detector agent.

Listing 1: Two-card combo query.

```
MATCH (c1:Card)-[:ENABLES_COMBO]->(combo:Combo)<-[:ENABLES_COMBO]-(c2:Card)
WHERE c1.name IN $library AND c2.name IN $library AND c1 <> c2
```

Comprehensive Rules concept	CR section	Implementing module
Turn structure / phases / steps	500–514	`src/engine/phases.py`
APNAP priority & priority loop	116–117	`src/engine/priority_loop.py`
The stack & spell resolution	405–608	`src/engine/stack.py`
Layer system (continuous effects)	613	`src/engine/continuous_effects.py`
Replacement effects	614–616	`src/engine/replacement_effects.py`
Triggered abilities	603	`src/engine/triggered_abilities.py`
State-based actions	704	`src/engine/state_based_actions.py`
Combat (declare, block, damage steps)	506–510	`src/engine/combat.py`
Mana & cost payment	106, 601–602	`src/engine/mana.py`
London mulligan	103.5	`src/engine/game_setup.py`
Cleanup discard to hand size	514	`src/engine/phases.py::cleanup_step`
Loss from empty library on draw	104.3c, 704.5b	`src/engine/state_based_actions.py`
Legal-action enumeration	— (engine-internal)	`src/engine/rules_engine.py`

Table 2: Mapping from Comprehensive-Rules concepts to the implementing engine module. Every cell is a real path; auditing a rule reduces to opening that file. The legal-action enumerator (`rules_engine.py`) is the only point of contact between the engine and any agent.

```
RETURN combo.name, combo.win_condition, c1.name, c2.name
ORDER BY combo.cmc_total ASC LIMIT 10;
```

4.3 World model

Let $s_t \in \mathcal{S}$ be the game state at decision step t and $g \in \mathcal{G}$ the player's KG context (deck, archetype tag, known opponent information). The world model decomposes as:

$$z_t = \mathrm{Enc}_V(s_t) \qquad \text{(state encoder, Set-Transformer + VAE)} \tag{7}$$

$$g_t = \mathrm{Enc}_{\mathrm{KG}}(g, s_t) \qquad \text{(KG context encoder)} \tag{8}$$

$$h_t, \pi_t^M = \mathrm{LSTM}_M([z_t; g_t], h_{t-1}) \qquad \text{(MDN-LSTM dynamics)} \tag{9}$$

$$\hat{z}_{t+1} = \mathrm{JEPA}([z_t; g_t; a_t]) \qquad \text{(predictor head)} \tag{10}$$

$$a_t = C(z_t, g_t, h_t) \qquad \text{(controller).} \tag{11}$$

Enc_V produces $z_t \in \mathbb{R}^{256}$. The VAE term acts as a regulariser; the JEPA head predicts the *encoder output* z_{t+1}, not a reconstruction, and is trained with $\mathrm{MSE}(z_{t+1}, \hat{z}_{t+1}) + \beta\ \mathrm{KL}(q_\phi \,\|\, p)$. Surprise at step $t+1$ is the predictor residual $\|\hat{z}_{t+1} - z_{t+1}\|_2$, which seeds curiosity bonuses and KG-enrichment triggers.

Concrete tensor shapes. For reproducibility we record the default dimensions used in `src/world_model/world_model.py`: state tokens $x_t \in \mathbb{R}^{B\times N\times 64}$ with $N \le 256$ (zero-padded with an attention mask), latent $z_t \in \mathbb{R}^{B\times 256}$, KG context $g_t \in \mathbb{R}^{B\times 64}$ produced by mean-pooling a two-layer GraphSAGE over the cards in the player's library, LSTM hidden $h_t \in \mathbb{R}^{B\times 512}$, MDN with $K = 5$ Gaussian components, and policy logits $\pi_t^M \in \mathbb{R}^{B\times A}$ where A is the size of the legal-action set at step t (variable; the controller masks illegal entries to $-\infty$ before softmax). The JEPA target tensor matches z_{t+1} (no separate decoder); the controller value head outputs a scalar $V_t \in \mathbb{R}^B$. The checkpoint loader (`WorldModel.load`) is shape-tolerant: missing keys fall back to freshly-initialised weights, which lets us swap in checkpoints from earlier minor versions during ablation studies.

4.4 Active-inference LLM-fusion agent

The fused agent maintains a belief b_t over hidden state (opponent's hand and library) using a particle filter constrained by the published decklist, and selects actions by minimising expected free energy:

$$\mathcal{F}(a) \;=\; \underbrace{-\,\mathbb{E}_{q(s'|a)}[R(s')]}_{\text{pragmatic}} \;-\; \underbrace{\mathrm{H}[q(s'|a)] - \mathbb{E}[\mathrm{H}[q(s'|s,a)]]}_{\text{epistemic (information gain)}} \;-\; \alpha \,\log p_{\mathrm{LLM}}(a \mid \mathrm{prompt}_t). \tag{12}$$

The LLM term provides a calibrated prior over the legal-action set; the controller of Section 4.3 provides R-estimates by short rollouts in the dream; and the KG provides hard combinatorial constraints (e.g., "no spell with mana cost > 3 in a deck whose curve peaks at 2").

Worked example. Consider a mid-game decision with three legal actions: a_1 = cast *Counterspell* on a stack object, a_2 = pass priority, a_3 = activate a card-draw ability. With pragmatic-value horizon $H = 3$ controller rollouts the model returns $R(a_1) = +0.42$, $R(a_2) = -0.05$, $R(a_3) = +0.18$ (utilities normalised to $[-1, 1]$ over expected life-total swing). The particle filter has high posterior entropy on the opponent's hand; the epistemic term $\mathrm{H}[q(s'|a)] - \mathbb{E}[\mathrm{H}[q(s'|s,a)]]$ peaks for a_3 (drawing reveals our top-of-library and lets us tighten the belief over our own future draws), giving an information-gain bonus of $+0.11$ versus ≤ 0.02 for the other actions. The LLM prior, asked "Given this board state, rank the three actions," returns log-probabilities $(-0.6, -1.4, -1.0)$. With weights $(\alpha_R, \alpha_E, \alpha_{\mathrm{LLM}}) = (1.0, 0.5, 0.3)$ the EFE scores are $\mathcal{F}(a_1) = -0.42 - 0.5 \cdot 0.02 - 0.3 \cdot (-0.6) = -0.25$, $\mathcal{F}(a_2) = +0.05 - 0.01 + 0.42 = +0.46$, $\mathcal{F}(a_3) = -0.18 - 0.055 + 0.30 = +0.065$. The agent picks a_1 (lowest free energy), matching the heuristic of countering rather than passing on a known threat. The same mechanism, with epistemic weight α_E raised to 1.0, would have selected a_3 — the explore–exploit trade-off becomes a single hyperparameter.

4.5 Self-play training loop

We use a champion-vs-challenger promotion scheme:

Algorithm 1 Champion-vs-Challenger Self-Play (Phase A.5).

```
Initialise pool 𝒫 with K randomly-seeded agents.
Pick champion c* ← arg max_{c∈𝒫} ELO(c).
for round = 1, ..., R do
    Sample challenger c ~ 𝒫 \ {c*}.
    Play N games c vs c*, collect trajectories τ.
    Update ELO for both; train all pool members on τ.
    wr ← challenger win rate over the N games.
    if wr ≥ θ then
        Promote: c* ← c; bump ELO by +50, demote former by −20.
    end if
end for
```

θ is the promotion threshold (default 0.55). The loop is implemented in `src/training/rl_trainer.py` and is exercised end-to-end by `scripts/train_pipeline.py`.

4.6 Agent zoo: algorithmic specification

The framework provides eight agents, all satisfying the same one-method contract `decide_action(state, legal_actions)` $\rightarrow$ `action` (see `src/agents/base_agent.py`). They span the full spectrum

from purely combinatorial baselines to the V+M+C+JEPA+KG+LLM fusion of Section 4.3. We summarise each agent below; the engineering reference (exact class signatures, configuration knobs, fallback paths, and test pointers) is the companion technical report `docs/AGENTS_TECH_REPORT.md`.

Random (π^{rand}). A weighted sampler over legal actions with fixed type-class weights $w_{\text{land}} = 25$, $w_{\text{cast}} = 10$ (boosted to 30 for casts from the command zone), $w_{\text{atk}} = 8$, $w_{\text{pass}} = 1$. Used as a baseline floor; not deterministic per-instance because it shares the process-global RNG.

Heuristic (π^{heur}). Deterministic priority cascade

$$a^\star = \arg\max_{a \in \mathcal{A}_{\text{legal}}} \text{prio}(a), \qquad \text{prio} : \begin{cases} \textsc{play\_land} \mapsto 100 \\ \textsc{cast\_spell} \mapsto 90 \\ \textsc{declare\_attackers} \mapsto 80 \\ \textsc{declare\_blockers} \mapsto 70 \\ \textsc{activate\_ability} \mapsto 60 \\ \textsc{pass} \mapsto 1 \\ \textsc{concede} \mapsto 0, \end{cases}$$

with three tactical refinements: (i) counter-spell discipline filters `counter target` casts unless an opponent spell tops the stack, (ii) when `prefer_aggressive` is set the agent always picks an attack at the top tier, (iii) on the blocking step it chump-blocks the largest unblocked attacker with the smallest blocker. Tie-breaking uses a seeded `random.Random`, making it the strongest fully-reproducible baseline.

KG-Heuristic ($\pi^{\text{KG+heur}}$). Inherits π^{heur} but, when more than one CAST_SPELL is legal, re-ranks them through the knowledge graph. Let H be the agent's hand and B its battlefield. For each candidate cast c,

$$\text{score}_{\text{KG}}(c) \;=\; 10 \cdot \mathbf{1}\big[c \in \text{NearComboMissingPieces}(H \cup B)\big] \;+ \sum_{p \in B} \text{strength}\big(\text{KG-edge}(c, p)\big),$$

where the indicator is computed by a single Cypher round-trip (`kg.detect_near_combos`) and synergy strength comes from the `HAS_SYNERGY` edges of `??` 1. The best-scoring cast replaces the cast subset in the legal-action list and the parent priority logic resumes, so PLAY_LAND can still beat CAST_SPELL. On any KG exception the agent latches `kg_disabled = True` and silently degrades to π^{heur}.

Ollama LLM (π^{LLM}). The LLM is consulted as a discrete posterior over the legal-action set, never as a free-form generator. Concretely, for each decision we serialise the board state s_t (life totals, hand sizes, lands, creatures) and an enumerated list $\{(i, a_i)\}_{i=0}^{|\mathcal{A}_{\text{legal}}|-1}$, prompt the model to "respond with ONLY the option number", parse an integer $\hat{\imath}$ from the reply, and return $a_{\hat{\imath}}$. Formally,

$$\hat{\imath} = \pi_{\text{LLM}}(\text{prompt}_t), \quad a^\star = a_{\text{clip}(\hat{\imath},\, 0,\, |\mathcal{A}_{\text{legal}}|-1)}.$$

Illegal actions are unrepresentable by construction. We use Gemma 4-edge (`gemma4:e2b`) at temperature 0.2 via Ollama for a $\sim$1–4 s budget per decision. On any HTTP or parse failure the agent falls back to π^{rand} and records the event in a per-game stats dictionary.

Active Inference (π^{AI}). Maintains a particle-filter belief b_t over the opponent's hand and library top conditioned on the published decklist (`src/agents/opponent_model.py`). Action selection minimises the discrete-state expected free energy

$$\mathcal{F}(a) \;=\; -\,\mathbb{E}_{q(s'|a,b_t)}[R(s')] \;-\; \mathrm{IG}(a,b_t), \qquad \mathrm{IG}(a,b_t) \;=\; \mathrm{H}[q(s'|a,b_t)] - \mathbb{E}_{q(s|b_t)}[\mathrm{H}[q(s'|s,a)]],$$

exposed as `ActiveInferenceModule.rank_actions(legal, state)`. A tactical override re-ranks DECLARE_ATTACKERS actions by target threat $\max(0, 40-\text{life})+2\cdot\text{commander\_dmg\_taken}$, and a counter-spell-aware filter drops non-mana CAST_SPELL actions when the opponent model places probability > 0.5 on a counter-spell being held.

World-Model agent (π^{WM}). Plays through the V+M+C+JEPA stack of Section 4.3. Every turn,

$$z_t = \mathrm{Enc}_V(s_t), \quad g_t = \mathrm{Enc}_{\mathrm{KG}}(g, s_t), \quad (h_t, c_t) = \mathrm{LSTM}_M([z_t; g_t], (h_{t-1}, c_{t-1})).$$

Two action-selection modes are available: a fast *direct policy* that calls the controller $C(z, h, A)$ and returns its argmax (or a temperature-scaled sample if `deterministic = False`), and a slower *dream search* that, for each candidate a_i, runs K rollouts of depth D in latent space using M, scores each rollout by the controller's value head $V(z)$, and picks $\arg\max_i \frac{1}{K}\sum_k V(z_i^{(k,D)})$. Defaults are $K = 8, D = 10$. The hidden state (h, c) is persisted across turns so the dynamics model sees the full game-step trajectory.

LLM-Fusion agent (π^{fuse}). The strongest agent: a calibrated weighted sum of four scalar signals per legal action,

$$\mathrm{score}(a) \;=\; \alpha_h\, s_{\mathrm{heur}}(a) \;+\; \alpha_g\, s_{\mathrm{KG}}(a) \;+\; \alpha_w\, s_{\mathrm{WM}}(a) \;+\; \alpha_\ell\, \log p_{\mathrm{LLM}}(a), \quad a^\star = \arg\max_a \mathrm{score}(a),$$

with default weights $(\alpha_h, \alpha_g, \alpha_w, \alpha_\ell) = (0.15, 0.15, 0.40, 0.30)$. The LLM signal is skipped when one action dominates the heuristic-plus-WM signal by more than `obvious_threshold` $= 0.8$, which saves roughly 80% of LLM calls in practice. Setting any single weight to zero recovers the corresponding ablation; the canonical four-arm study (LLM-only, WM-only, KG-only, all four) is wired up by `scripts/run_matchups.py`.

Algorithm 2 LLM-Fusion decision (one priority window).

Require: state s_t, legal actions $\mathcal{A}$, world model (Enc_V, M, C), KG $\mathcal{G}$, opponent model b_t, weights $(\alpha_h, \alpha_g, \alpha_w, \alpha_\ell)$

$\mathbf{s}_h \leftarrow \mathrm{HeuristicScores}(s_t, \mathcal{A})$ ▷ $\sim$50 µs
$\mathbf{s}_g \leftarrow \mathrm{KGScores}(s_t, \mathcal{A})$ ▷ combo + synergy bonuses
$z_t \leftarrow \mathrm{Enc}_V(s_t)$; $\mathbf{s}_w \leftarrow \mathrm{DreamSearch}(z_t, h_t, \mathcal{A}; K, D)$
if $\mathrm{spread}(\mathbf{s}_h + \mathbf{s}_w) > \tau_{\mathrm{obvious}}$ **then**
    $\mathbf{s}_\ell \leftarrow \mathbf{0}$ ▷ skip the LLM call
else
    $\mathbf{s}_\ell \leftarrow \log p_{\mathrm{LLM}}(\cdot \mid \mathrm{prompt}(s_t, \mathcal{A}))$
end if
return $a^\star \leftarrow \mathcal{A}\big[\arg\max_i(\alpha_h \mathbf{s}_{h,i} + \alpha_g \mathbf{s}_{g,i} + \alpha_w \mathbf{s}_{w,i} + \alpha_\ell \mathbf{s}_{\ell,i})\big]$

Reasoning traces. Every agent writes a `ReasoningTrace` dict to `self.last_reasoning` before returning. The orchestrator flushes these to `runs/<out>/logs/game_NNNN.jsonl`, one line per decision, schema {turn, phase, player_id, agent_kind, rationale, legal_action_count, chosen_index, top_candidates, scores, beliefs}. This is the canonical telemetry format consumed by the post-hoc analysis notebooks and by the Streamlit visualisation panel discussed in Section 8.

5 Implementation

The codebase is organised into eight top-level packages under `src/`: `engine/` (Comprehensive-Rules engine), `knowledge/` (Neo4j, n10s, GraphRAG, GraphSAGE), `world_model/` (V, M, C, JEPA, KG-encoder, tokeniser), `agents/` (LLM, fusion, active-inference, opponent-model), `orchestrator/` (LangGraph runner, priority loop), `training/` (RL trainer, self-play, rewards, benchmarks), `judge/` (RAG over Comprehensive Rules) and `integrations/` (Scryfall, decklist loaders, SQLite cache). External services (Neo4j, Ollama) are orchestrated via `docker-compose`. The default LLM is `gemma4:e2b` (Gemma 4 edge variant) for low-end hardware compatibility, swappable to any Ollama-served model.

Reproducibility. All randomness flows through a single `numpy.random.Generator` seeded by `config.seed`. Decklists are committed alongside the code under `examples/`. The pipeline (`scripts/train_pipeline.py`) has seven stages (infra check, KG build, GNN embedding, self-play, JEPA, dream evaluation, model evaluation) each individually re-runnable.

6 Preliminary Evaluation

We report preliminary results; a full benchmark suite (Section 8) is in development. Current evidence is of three kinds.

(i) Engine correctness. The repository contains ∼30 pytest modules covering combat, the stack, triggered/static/replacement abilities, and end-to-end games. The whole-suite runtime is dominated by integration tests; unit tests under `tests/test_engine/` and `tests/test_knowledge/` pass on Python 3.11 and 3.12.

(ii) Knowledge-graph coverage. The OWL ontology v1.1 declares 17 top-level classes and 24 object/data properties. After Scryfall ingestion of a representative Standard environment (∼5,000 cards) the graph contains the cards, their oracle-text SBERT embeddings, and combo edges imported from Commander Spellbook.

(iii) Self-play promotion loop. The promotion loop of Algorithm 1 is exercised by two unit tests (`tests/test_rl_trainer.py`); both pass. End-to-end promotion behaviour — whether champions stabilise after $\mathcal{O}(10)$ rounds of $N = 20$ games — is the next planned experiment and is not reported here.

We deliberately do not claim "state of the art" numbers: we know of no comparable public baseline that plays full-Comprehensive-Rules MTG, and the contribution of this report is the system itself, plus the ontology and the released self-play infrastructure.

7 Discussion and Limitations

Scope. The current engine targets a Standard subset and an extended Commander subset; banding, phasing, planeswalker loyalty abilities, and exotic targeting restrictions remain partial. The ontology v1.1 covers core mechanics but does not yet encode every keyword from every set, and we do not currently extend it automatically per Standard rotation — a known limitation that motivates planned integration with the OntologyExtender tool (Section 8).

Compute. Defaults are tuned for a single-GPU workstation: $z \in \mathbb{R}^{256}$, the LSTM hidden size is 512, the JEPA predictor is a 4-layer pre-norm Transformer, and the LLM is `gemma4:e2b`. Larger pretrained encoders or stronger LLMs (e.g., `llama-3.1-70b`) would likely improve play strength but were not the focus of this report.

Active inference vs. RL. Active inference here is implemented at the *decision* level (action selection conditional on world-model rollouts), not the *learning* level (no free-energy objective for the model itself). This is a deliberate choice: the world model is trained by the standard MSE+KL JEPA objective, which we found to be more compute-efficient than full Bayesian model-evidence maximisation.

LLM grounding. The LLM never produces game actions directly. It produces a posterior over the legal-action set provided by the engine; illegal actions are masked. This eliminates the dominant failure mode of LLM-only MTG bots (hallucinated cards, illegal plays) at the cost of a small expressivity loss.

8 Future Work

The following items are tracked in `IMPLEMENTATION_PLAN.md` and constitute the near-term roadmap:

1. **Strategy-aware and agent-driven mulligan policy.** The current keep/bottom heuristic is a single deterministic land-count rule shared by all agents. We are moving the decision through the agent interface (`MTGAgent.decide_mulligan`, `MTGAgent.bottom_cards`) so that aggressive, control, combo, and reactive strategies—and ultimately learned policies—can evaluate their own opening hands. JEPA-conditioned mulligan selection (rolling out a few imagined turns from each candidate hand) is the longer-term target.
2. **Priority-loop driven instants in the sync simulator.** The sync `GameSimulator` currently advances combat without exposing an instant-speed window to the non-active player; threading the existing `priority_loop` into combat steps will let counterspells and combat tricks fire mid-attack.
3. **Benchmark suite.** A `BenchmarkSuite` class with deterministic-replay seeds, four to six standard archetype decklists, baseline agents (Random, Heuristic, LLM-only, MCTS), and a per-game / per-agent metrics CSV.
4. **Ablation matrix.** Toggleable KG / dream / JEPA / opponent-model components with paired-game ELO comparison.
5. **Neural reasoner training.** A supervised + RL fine-tune of a graph-conditioned reasoner (`NeuralReasonerAgent`) replacing the SBERT card embedding with end-to-end GraphSAGE embeddings.
6. **Parallel self-play.** `asyncio.gather`-based parallel game execution to lift trajectory throughput, plus restoring `pytest-asyncio` so the async runner is exercised in CI.
7. **Belief-state visualisation.** JSON export and a Streamlit panel for the particle-filter posterior over the opponent's hand.
8. **Commander politics.** Threat assessment across $N{>}2$ opponents, including a multi-agent extension of the active-inference objective, and the Commander variant of the mulligan (free first mulligan).
9. **Ontology evolution.** Wire the framework into the OntologyExtender tool (Data Science Lab FHSWF, 2024) so the OWL schema is updated semi-automatically per Standard rotation, with humans in the loop on debated axioms. At present the ontology is a static, hand-curated v1.1 artefact.

9 Conclusion

We have presented a research framework that approaches Magic: The Gathering not as "a harder Atari" but as a domain that fundamentally requires the combination of symbolic verification, structured knowledge, and learned imagination. The released codebase — engine, ontology, knowledge graph, V+M+C+JEPA world model, and active-inference LLM agent — is, to our knowledge, the first open system that puts these pieces together end-to-end on full Comprehensive-Rules MTG. We hope it is useful both as a benchmark for neuro-symbolic agents and as a sandbox for ideas in active inference, JEPA, and GraphRAG.

Reproducibility statement. All code, the OWL ontology, sample decklists, training scripts, and this report are available under MIT at `https://github.com/cuteredpwnda/opposition-agents-playing`

Acknowledgments

The authors thank the open-source maintainers of `open-mtg`, `Forge`, `XMage`, `Scrython`, `LangChain`, `LangGraph`, `Neo4j`, `n10s`, `APOC`, and `Ollama`, on whose work this project relies.

References

D. Ha and J. Schmidhuber. World Models. *NeurIPS*, 2018.

Y. LeCun. A Path Towards Autonomous Machine Intelligence. *OpenReview*, 2022.

M. Assran et al. Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture. *CVPR*, 2023.

D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi. Dream to Control: Learning Behaviors by Latent Imagination. *ICLR*, 2020.

D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap. Mastering Diverse Domains through World Models. *arXiv:2301.04104*, 2023.

J. Schmidhuber. On Learning to Think: Algorithmic Information Theory for Novel Combinations of RL Controllers and Recurrent Neural World Models. *arXiv:1511.09249*, 2015.

K. Irie et al. A Modern Self-Referential Weight Matrix that Learns to Modify Itself. *ICML*, 2022.

K. Friston. The Free-Energy Principle: A Unified Brain Theory? *Nature Reviews Neuroscience*, 11(2):127–138, 2010.

N. Sajid, P. J. Ball, T. Parr, and K. J. Friston. Active Inference: Demystified and Compared. *Neural Computation*, 33(3):674–712, 2021.

G. Tesauro. Temporal Difference Learning and TD-Gammon. *Communications of the ACM*, 38(3):58–68, 1995.

D. Silver et al. Mastering the Game of Go without Human Knowledge. *Nature*, 550(7676):354–359, 2017.

D. Silver et al. A General Reinforcement Learning Algorithm that Masters Chess, Shogi, and Go through Self-Play. *Science*, 362(6419):1140–1144, 2018.

J. Schrittwieser et al. Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model. *Nature*, 588(7839):604–609, 2020.

O. Vinyals et al. Grandmaster Level in StarCraft II Using Multi-Agent Reinforcement Learning. *Nature*, 575(7782):350–354, 2019.

M. Moravčík et al. DeepStack: Expert-Level Artificial Intelligence in Heads-up No-Limit Poker. *Science*, 356(6337):508–513, 2017.

N. Brown and T. Sandholm. Superhuman AI for Heads-up No-Limit Poker: Libratus Beats Top Professionals. *Science*, 359(6374):418–424, 2018.

P. I. Cowling et al. Ensemble Determinization in Monte Carlo Tree Search for the Imperfect Information Card Game Magic: The Gathering. *IEEE Transactions on Computational Intelligence and AI in Games*, 2012.

W. L. Hamilton, R. Ying, and J. Leskovec. Inductive Representation Learning on Large Graphs. *NeurIPS*, 2017.

D. Edge et al. From Local to Global: A Graph RAG Approach to Query-Focused Summarisation. *arXiv:2404.16130*, 2024.

N. F. Noy and D. L. McGuinness. Ontology Development 101: A Guide to Creating Your First Ontology. Stanford Knowledge Systems Laboratory, 2001.

H. Knublauch and D. Kontokostas (eds.). Shapes Constraint Language (SHACL). *W3C Recommendation*, 2017.

Neo4j Labs. Neosemantics (n10s): RDF and Semantics Toolkit for Neo4j. https://neo4j.com/labs/neosemantics/.

Data Science Lab, Fachhochschule Südwestfalen. OntologyExtender: A Multi-Agent Human-in-the-Loop Framework for Ontology Evolution. GitHub repository (work in progress), 2024. https://github.com/DataScienceLabFHSWF/OntologyExtender.